Unit-2 mini project

STUDENT RECORD AND ACADEMIC MANAGEMENT SYSTEM

Student Name	Pruthviraj Somvanshi
PRN	2126UDSM1063
Department	AIDS
Project focus	Store, search, update, delete and report student academic records.

Project repository

GitHub - pruthvis25/student-record-management-system · GitHub

Python concepts: Strings, lists, tuples, sets, dictionaries, loops, functions and validation.

Included: Problem analysis, data-structure selection, algorithms, flowcharts, pseudocode, code, test cases and README notes.

Prepared from the assignment brief supplied.

1. Problem Understanding and Analysis

Problem statement: Develop a menu-driven Python application that stores student details and academic results, then lets a user add, search, update, delete, display and analyse those records. The system must reject duplicate roll numbers and handle invalid or empty inputs safely.

Objective: Use Python collections in a connected program and make student data easy to organize, retrieve and summarize.

Inputs: Roll number, registration number, name, department, date of birth, contact details, subjects, marks, attendance, clubs and menu choice.

Outputs: Student profile, confirmation/error messages, overall average marks, highest scorer, department-wise student list, total student count and collection summaries.

Relationships: Each roll number identifies one student. A student can have several subjects and marks; each mark corresponds to a subject at the same list position. Contact information belongs to that student. A student may join zero or more clubs.

Assumptions: Roll numbers are unique text values; marks and attendance are percentages from 0 to 100; each student has 1–10 subjects; dates use YYYY-MM-DD; records remain in memory while the program is running (no database/file persistence is requested in the brief).

2. Data Structure Selection

Structure	How it is used	Reason
String	Name, department, address, email, phone and subject names.	Stores readable text.
List	Subjects and marks for one student.	Keeps order; subjects and marks align by index.
Tuple	Identity tuple: (roll, registration number, date of birth).	Groups identity fields in a fixed sequence.
Set	Unique departments, subjects and clubs across all students.	Automatically avoids duplicate values.
Dictionary	Main students mapping and each student profile/contact details.	Fast lookup by roll number and clear key-value fields.

Main data model: students[roll] = {identity: tuple, name: string, department: string, subjects: list, marks: list, attendance: number, contact: dictionary, clubs: set}

3. Algorithms

Add student

1. Start
2. Read roll number
3. If roll already exists, show duplicate warning and stop this operation
4. Read and validate profile fields, date, subjects, marks and attendance
5. Build profile using tuple, lists, set and dictionaries
6. Store profile in students[roll]; update summary sets
7. Show success message
8. End

Search student

9. Start

10. Read roll number
11. Look up students[roll]
12. If found, display profile; otherwise show not found
13. End

Update record

14. Start
15. Read roll number
16. Check whether record exists
17. If not found, show message and stop
18. Choose a supported field
19. Read and validate new value
20. Update the selected field
21. Show confirmation
22. End

Delete record

23. Start
24. Read roll number
25. Check whether record exists
26. If not found, show message and stop
27. Ask for confirmation
28. If confirmed, delete record and rebuild summary sets
29. Show result
30. End

Display records

31. Start
32. Check whether students is empty
33. If empty, show no-records message
34. Otherwise display each student in sorted roll order
35. End

Calculate average

36. Start
37. Check whether records and marks exist
38. Add all marks and count all marks
39. Average = total marks / number of marks
40. Display average
41. End

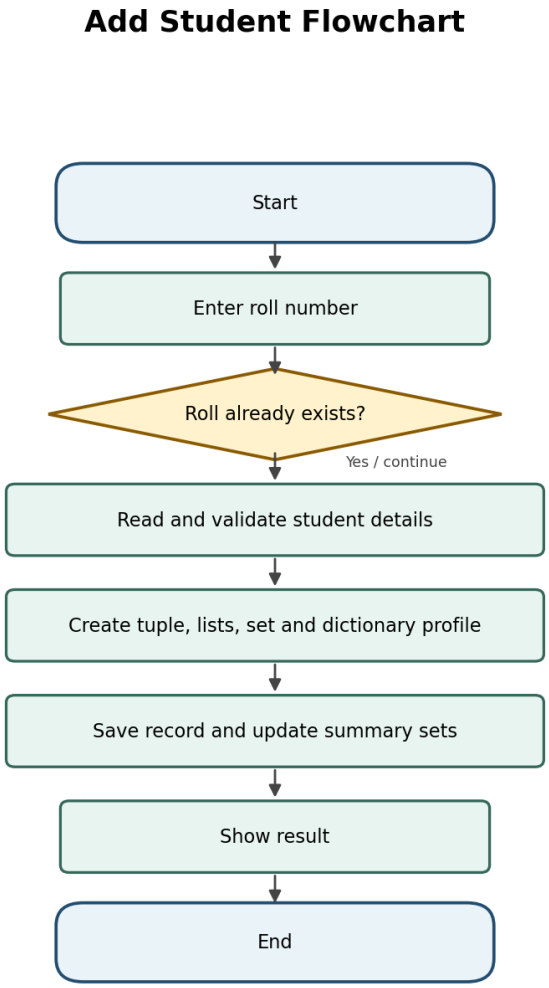
Generate reports

42. Start
43. Show report menu option
44. For highest scorer, compare each student average
45. For department report, filter by department
46. For count, use number of records
47. Display requested report
48. End

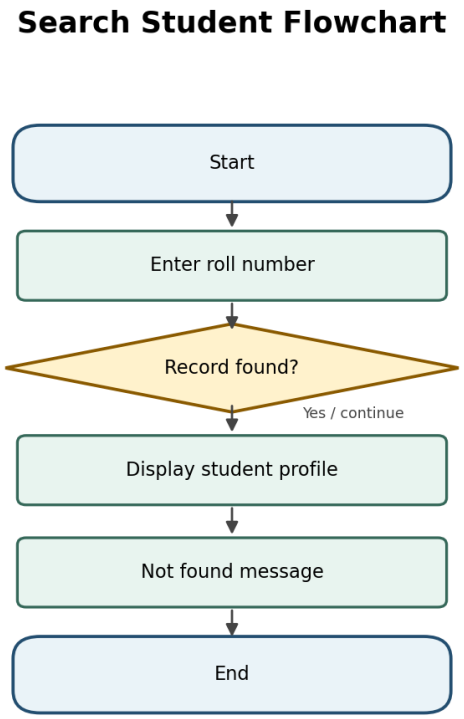
4. Flowcharts

Rounded terminator = start/end; rectangle = process; diamond = decision. The diagrams have been redrawn in a consistent format for readability.

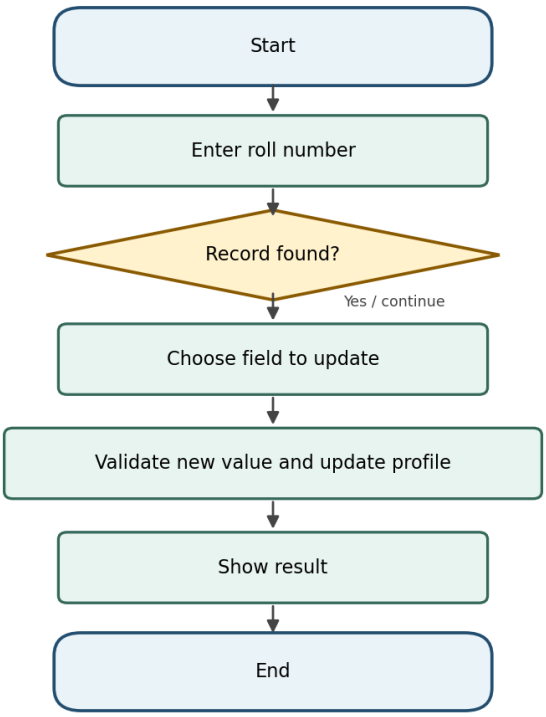
Add Student Flowchart



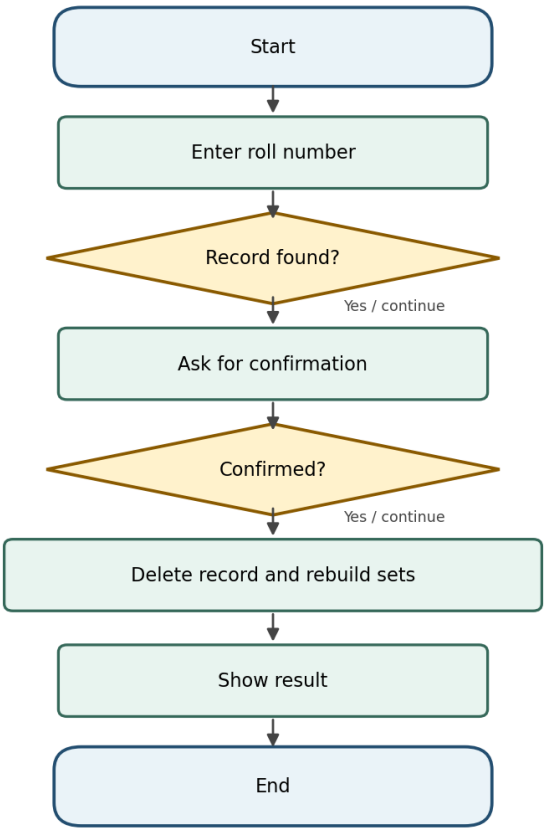
Search Student Flowchart



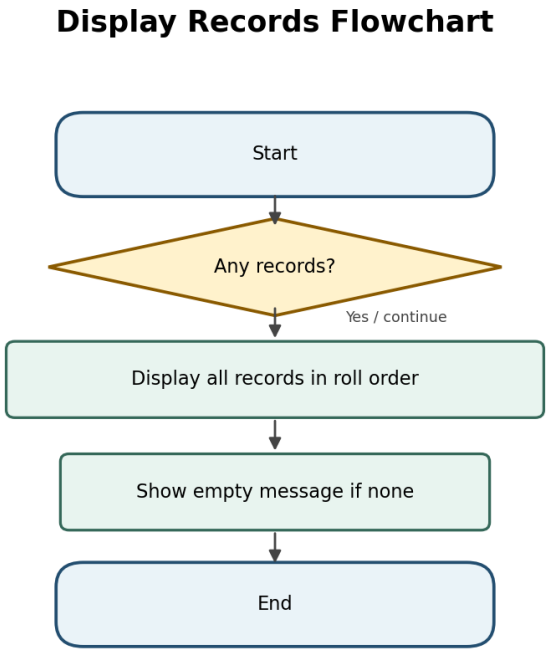
Update Record Flowchart



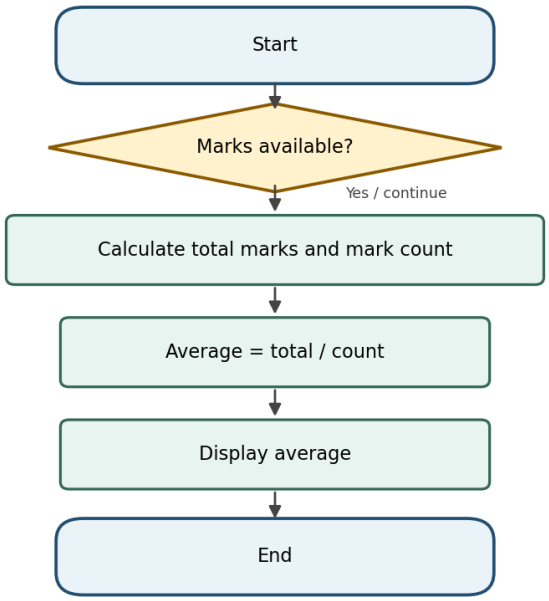
Delete Record Flowchart



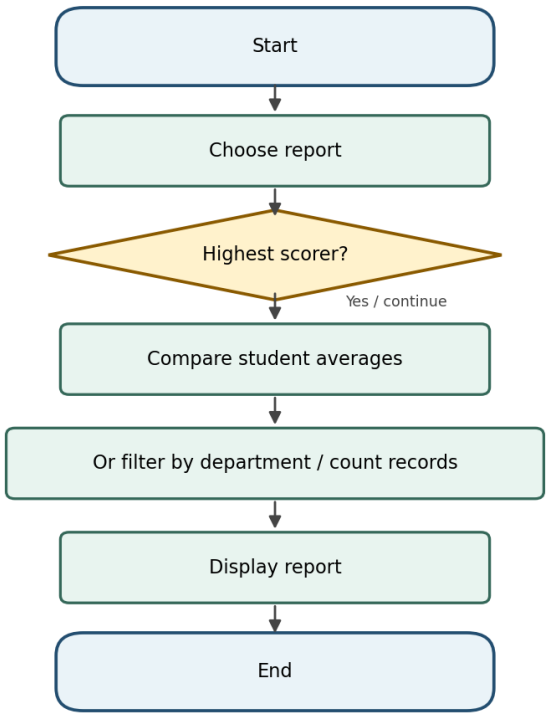
Display Records Flowchart



Calculate Average Flowchart



Generate Reports Flowchart



5. Pseudocode

```
ADD STUDENT
BEGIN
    INPUT roll
    IF roll exists in students THEN
        DISPLAY "Duplicate roll number"; RETURN
    ENDIF
    INPUT and validate student identity and contact fields
    INPUT subjects and corresponding marks
    INPUT attendance and clubs
    profile ← dictionary containing tuple, lists, contact dictionary and club set
    students[roll] ← profile
    UPDATE department, subject and club sets
    DISPLAY success
END

SEARCH STUDENT
BEGIN
    INPUT roll
    IF roll exists in students THEN
        DISPLAY students[roll]
    ELSE
        DISPLAY "Record not found"
    ENDIF
```

```
END

UPDATE / DELETE
BEGIN
    INPUT roll
    IF roll does not exist THEN DISPLAY "Not found"; RETURN
    For UPDATE: choose field, validate value, update profile
    For DELETE: ask confirmation; if yes, remove record and rebuild sets
    DISPLAY result
END

AVERAGE / REPORTS
BEGIN
    IF no records or no marks THEN DISPLAY suitable message
    ELSE
        total ← sum of all recorded marks
        count ← number of recorded marks
        overall_average ← total / count
        DISPLAY overall_average
    ENDIF
    For highest scorer, compare each student's average
    For department report, filter records by department
    For student count, display length of students
END
```

6. Python Source Code

The complete source code is included below for documentation. Use a Python 3.10 or later interpreter to run the program.

```
# Student Record and Academic Management System
# Python Collections: Strings, Lists, Tuples, Sets, Dictionaries
# Records are saved automatically in students.json.

import json
import os

DATA_FILE = "students.json"
students = []


def load_students():
    """Load saved records when the program starts."""
    global students
    if not os.path.exists(DATA_FILE):
        students = []
        return
    try:
        with open(DATA_FILE, "r", encoding="utf-8") as file:
            data = json.load(file)
        students = data if isinstance(data, list) else []
        print(f"Loaded {len(students)} saved student record(s).")
    except (json.JSONDecodeError, OSError):
        students = []
        print("Could not read saved records. Starting with an empty list.")


def save_students():
    """Save all records to a JSON file."""
    try:
        with open(DATA_FILE, "w", encoding="utf-8") as file:
            json.dump(students, file, indent=4, ensure_ascii=False)
    except OSError:
        print("Warning: records could not be saved.")
```

```
def find_student(roll_no):
    for student in students:
        if student["roll_no"].lower() == roll_no.lower():
            return student
    return None


def read_number(prompt, minimum=None, maximum=None, whole_number=False):
    while True:
        try:
            value = input(prompt).strip()
            number = int(value) if whole_number else float(value)
            if minimum is not None and number < minimum:
                print(f"Value must be at least {minimum}.")
                continue
            if maximum is not None and number > maximum:
                print(f"Value must not be more than {maximum}.")
                continue
            return number
        except ValueError:
            print("Invalid input. Please enter a valid number.")


def read_non_empty(prompt):
    while True:
        value = input(prompt).strip()
        if value:
            return value
        print("This field cannot be empty.")


def read_marks():
    return read_number("Enter marks (0-100): ", 0, 100)
```

```
def read_attendance():
    return read_number("Enter attendance percentage (0-100): ", 0, 100)


def add_student():
    print("\n--- Add Student ---")
    roll_no = read_non_empty("Enter Roll Number: ")
    if find_student(roll_no):
        print("This roll number already exists. Duplicate record not added.")
        return

    name = read_non_empty("Enter Student Name: ")
    department = read_non_empty("Enter Department: ")
    registration_no = input("Enter Registration Number (or press Enter to skip): ").strip()
    dob = input("Enter Date of Birth (or press Enter to skip): ").strip()
    address = input("Enter Address (or press Enter to skip): ").strip()
    email = input("Enter Email ID (or press Enter to skip): ").strip()
    phone = input("Enter Contact Number (or press Enter to skip): ").strip()

    subjects_text = input("Enter Subjects separated by commas: ").strip()
    subjects = [subject.strip() for subject in subjects_text.split(",") if subject.strip()]

    marks = read_marks()
    attendance = read_attendance()

    # Tuple example: roll number and registration number are grouped as identifiers.
    identifiers = (roll_no, registration_no)

    student = {
        "roll_no": identifiers[0],
        "registration_no": identifiers[1],
        "name": name,
        "department": department,
        "dob": dob,
        "address": address,
```

```
        "email": email,
        "phone": phone,
        "subjects": subjects,
        "marks": marks,
        "attendance": attendance
    }

    students.append(student)
    save_students()
    print("Student added and saved successfully!")


def print_student(student):
    print("-" * 35)
    print("Roll Number:", student.get("roll_no", ""))
    print("Registration Number:", student.get("registration_no", ""))
    print("Name:", student.get("name", ""))
    print("Department:", student.get("department", ""))
    print("Date of Birth:", student.get("dob", ""))
    print("Address:", student.get("address", ""))
    print("Email:", student.get("email", ""))
    print("Contact:", student.get("phone", ""))
    print("Subjects:", ", ".join(student.get("subjects", [])))
    print("Marks:", student.get("marks", 0))
    print("Attendance:", str(student.get("attendance", 0)) + "%")


def display_students():
    print("\n--- Display All Students ---")
    if not students:
        print("No student records found.")
        return
    for student in students:
        print_student(student)
    print("-" * 35)
    print("Total students:", len(students))
```

```
def search_student():
    print("\n--- Search Student ---")
    roll_no = input("Enter Roll Number to search: ").strip()
    student = find_student(roll_no)
    if student:
        print_student(student)
    else:
        print("Student not found.")


def update_student():
    print("\n--- Update Student ---")
    roll_no = input("Enter Roll Number to update: ").strip()
    student = find_student(roll_no)
    if not student:
        print("Student not found.")
        return

    print("Press Enter to keep the existing value.")
    fields = [
        ("name", "Name"),
        ("department", "Department"),
        ("registration_no", "Registration Number"),
        ("dob", "Date of Birth"),
        ("address", "Address"),
        ("email", "Email ID"),
        ("phone", "Contact Number")
    ]
    for key, label in fields:
        current = student.get(key, "")
        value = input(f"{label} [{current}]: ").strip()
        if value:
            student[key] = value
```

```
    subjects_text = input(
        "Update subjects (comma-separated, or Enter to keep current): "
    ).strip()
    if subjects_text:
        student["subjects"] = [
            item.strip() for item in subjects_text.split(",") if item.strip()
        ]

    if input("Update marks? (y/n): ").strip().lower() == "y":
        student["marks"] = read_marks()
    if input("Update attendance? (y/n): ").strip().lower() == "y":
        student["attendance"] = read_attendance()

    save_students()
    print("Student record updated and saved.")


def delete_student():
    print("\n--- Delete Student ---")
    roll_no = input("Enter Roll Number to delete: ").strip()
    student = find_student(roll_no)
    if not student:
        print("Student not found.")
        return

    confirm = input(
        f'Confirm deletion of {student.get("name", "this student")}? (y/n): '
    ).strip().lower()
    if confirm == "y":
        students.remove(student)
        save_students()
        print("Student deleted.")
    else:
        print("Deletion cancelled.")
```

```
def average_marks():
    print("\n--- Average Marks ---")
    if not students:
        print("No student records found.")
        return
    average = sum(float(s.get("marks", 0)) for s in students) / len(students)
    print(f"Average Marks: {average:.2f}")


def highest_scorer():
    print("\n--- Highest Scorer ---")
    if not students:
        print("No student records found.")
        return
    highest = max(students, key=lambda s: float(s.get("marks", 0)))
    print_student(highest)


def list_by_department():
    print("\n--- List Students by Department ---")
    department = input("Enter Department to list: ").strip()
    matches = [
        student for student in students
        if student.get("department", "").lower() == department.lower()
    ]
    if not matches:
        print("No students found in this department.")
        return
    for student in matches:
        print_student(student)
    print("Students in this department:", len(matches))


def count_students():
    print("\n--- Count Students ---")
    print("Total student records:", len(students))
```

```
    if students:
        # Set keeps only unique department names.
        departments = {s.get("department", "") for s in students if s.get("department", "")}
        print("Number of different departments:", len(departments))
        print("Departments:", ", ".join(sorted(departments)))


def collections_demo():
    print("\n--- Python Collections Demonstration ---")
    # List: student records and subject lists.
    print("Student records are stored in a LIST.")
    print("Number of records:", len(students))

    # Tuple: immutable example.
    sample_tuple = ("Roll Number", "Registration Number", "Date of Birth")
    print("Example tuple:", sample_tuple)

    # Set: unique departments and subjects.
    departments = {s.get("department", "") for s in students if s.get("department", "")}
    subjects = {
        subject
        for s in students
        for subject in s.get("subjects", [])
    }
    print("Unique departments (set):", departments)
    print("Unique subjects (set):", subjects)

    # String examples.
    if students:
        name = students[0].get("name", "")
        print("First student's name in uppercase:", name.upper())
        print("First student's name length:", len(name))
    else:
        print("Add a student to see string examples.")
```

```
def main():
    load_students()
    while True:
        print("\n========================================")
        print(" STUDENT RECORD AND ACADEMIC MANAGEMENT")
        print("========================================")
        print("1. Add Student")
        print("2. Display All Students")
        print("3. Search Student")
        print("4. Update Student")
        print("5. Delete Student")
        print("6. Calculate Average Marks")
        print("7. Find Highest Scorer")
        print("8. List Students by Department")
        print("9. Count Students")
        print("10. Python Collections Demonstration")
        print("11. Exit")

        choice = input("Enter your choice (1-11): ").strip()

        if choice == "1":
            add_student()
        elif choice == "2":
            display_students()
        elif choice == "3":
            search_student()
        elif choice == "4":
            update_student()
        elif choice == "5":
            delete_student()
        elif choice == "6":
            average_marks()
        elif choice == "7":
            highest_scorer()
        elif choice == "8":
            list_by_department()
```

```
        elif choice == "9":
            count_students()
        elif choice == "10":
            collections_demo()
        elif choice == "11":
            save_students()
            print("Records saved. Thank you!")
            break
        else:
            print("Invalid choice. Enter a number from 1 to 11.")


if __name__ == "__main__":
    main()
```

7. Testing and Dry-Run Report

Test	Input / condition	Expected result
T1	Add a new roll number with valid data	Record saved; success message shown
T2	Add a roll number that already exists	Duplicate warning; old record unchanged
T3	Search for an existing roll number	Full profile is displayed
T4	Search for a missing roll number	Not-found message
T5	Enter text instead of marks	Invalid-number message; asks again
T6	Enter marks -1 or 101	Range warning; asks again
T7	Enter attendance 0 or 100	Accepted as boundary values
T8	Display records before adding any	No-records message
T9	Delete a record and confirm y	Record removed; summary sets rebuilt
T10	Choose an invalid menu option	Invalid-choice message; menu repeats
T11	Calculate average with valid marks	Sum of marks divided by number of marks
T12	Enter an invalid date	Date validation message; record not saved

Dry-run example: Marks [80, 90, 70] give total = 240, count = 3 and average = 80.00. If another student has average 85, the highest-scorer report shows that student.

Observations: Duplicate roll numbers are prevented; numerical inputs are checked; empty collections are handled; boundary marks and attendance from 0 to 100 are accepted; department matching ignores letter case.

Limitations: Data is stored in memory and disappears when the program exits. A real deployment would use file/database storage, stronger email/phone validation, access control and backups.

8. How to Run the Program

49. Install Python 3.10 or later (standard Python libraries only).
50. Save student_record_management_system.py to a folder.
51. Open Command Prompt / Terminal in that folder.

```
python student_record_management_system.py
```

52. Choose a number from the menu. Select 0 to exit.

9. README / Conclusion

Project name: Student Record and Academic Management System.

Purpose: Demonstrate how Python collections can work together to manage student academic records.

Features: Add, search, update, delete, display, calculate average, identify highest scorer, filter by department, count students, and list unique departments/subjects/clubs.

Conclusion: The program integrates strings, lists, tuples, sets and dictionaries in one application. Functions keep the code organized, loops support repeated menu use, and validation improves reliability. Test cases cover normal inputs, duplicates, missing records, invalid values, empty data and boundary values.

Before submission: Run the Python file and attach screenshots if your instructor requires them.